



UNIVERSITY OF
LEICESTER

School of Computing and Mathematical Sciences
CO7201 Individual Project

Final Report Template

**HomifyOne: An AI-Powered Personalisation
and Workflow Management Platform for
New-Build Residential Developments**

Student Name: Mrudhulaa Peechanary Vinodkumar

Student Email Id: mpv4@student.le.ac.uk

Student Number: 259025897

Project Supervisor: Dr Newman Lau

Principal Marker: Dr Zedong Zheng

Word Count: 7621

Date: 14/08/2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Mrudhulaa Peechanary Vinodkumar

Date: 14/08/2026

Contents

Chapter 1. Introduction	6
1.1. Motivation.....	6
1.2. Project Context.....	6
1.3. Target users and their problems	7
1.4. Aim and Objectives.....	7
1.5. Proposed Solution Overview	8
1.6. System Overview	8
1.6.1 Workflow Summary.....	8
Chapter 2. Background Research.....	10
2.1. New-build personalisation as a socio-technical workflow	10
2.2. Research Element 1: NLP-Based Semantic Recommendation Engine	10
2.3. Research Element 2: Conversational AI Assistant	11
2.4. Theoretical Foundation for Lifestyle and Behavioural Preference Modelling	12
2.5. Existing Platform Gap Analysis.....	12
Chapter 3. Requirements.....	17
3.1. Essential Requirements	17
3.2. Desirable Requirements	18
3.3. Optional Requirements	19
3.4. New Features Added.....	19
3.5. Non-functional requirements	20
3.6. Data and privacy requirements	20
3.7. Requirement changes	20
Chapter 4. Technical Specification	22
4.1. Tech Stack Overview	22
Chapter 5. Development Methodology and Project Management.....	24
5.1. Overview of Development Methodology	24
5.2. Prioritisation and milestones.....	24
5.3. Tools and Configuration Management	24
Chapter 6. System Design.....	25
6.1. System Design Architecture	25
6.2. Backend architecture.....	25
6.2.1. Middleware Stack	25
6.2.2. Backend Endpoints	26
6.3. Frontend architecture	27
6.4. Database Design.....	28
6.5. Use case Diagram	29

6.5.1. Specific Use Case - Product Ordering and Delivery to Site	30
6.6. Event Lifecycles.....	31
6.6.1. Plot state transition from creation to completion.....	31
6.6.2. Order and purchase-order lifecycles	32
6.6.3. Recommendation Flow	34
6.7. User interface and navigation design.....	34
6.8. Pagination and filtering.....	34
6.9. Calendar, messaging and notification design.....	34
6.10. Design of Auto-generation of Purchase Orders	34
6.11. Creation of Embeddings and storage for Recommendation Engine.....	34
6.12. Design of AI Assistant - Ask Homifyone	34
Chapter 7. Implementation.....	35
7.1. Authentication and Authorisation	35
7.2. Standard choices and paid upgrades	35
7.3. Questionnaire and profile building	35
7.4. Embedding preparation and FAISS Index	35
7.5. Hybrid scoring and explanations	35
7.6. Basket, credit and promotional pricing.....	35
7.7. Submission, approval and top-up orders.....	35
7.8. Purchase orders and supplier portal	35
7.9. Notifications, chat and scheduled work	35
7.10. AI assistant and summarisation of orders	35
7.11. Uploading invoice and summarisation.....	35
7.12. Administrator features and analytics	35
Chapter 8. Testing.....	36
8.1. Evaluation strategy.....	36
8.2. Unit Testing	36
8.3. Integration Testing	36
8.4. End To End Testing	36
8.5. Functional and Workflow Testing	36
8.6. API Testing	36
8.7. Evaluation of Recommendation output	36
Chapter 9. Critical Reflection	37
9.1. Key Achievements	37
9.2. Modified objectives and scope control	37
9.3. Technical challenges	37
9.4. Risk Control	37

9.5. Architectural decision reflection	38
9.6. Evaluation reflection	38
9.7. Project management reflection	38
9.8. Professional and personal learning	38
Chapter 10. Conclusion.....	39
Chapter 11. Recommendation for Future Work	40
Chapter 12. References	41
Chapter 13. Appendices	43

Chapter 1. Introduction

1.1. Motivation

The UK new-build housing is seen as a sector where the largest amount of financial commitment is made by buyers. Research shows that UK housebuilders have not put the consumer at the centre of the new-build experience [1]. The Commission for Architecture and the Built Environment (CABE) found that home buyers' opinions have a limited impact on what is offered to them [2]. One of the studies by Competition and Markets Authority (2024) has found that buyers often cannot make well-informed decisions at important points during the transaction process [3]. Studies in construction management confirm that buyers place a high value on the ability to select the choices and extras for their homes according to their preferences, yet the industry lacks the ability to capture and respond to those preferences on a large scale [4]. Despite the gradual digitisation efforts, the home personalisation arena is still fragmented. Existing platforms serve only one or two stakeholder groups at most. There has not been a commercial tool so far that brings Buyer, Developer, Supplier, and Administrator workflows into a single application nor one available for real-time multi-stakeholder collaboration in this domain [5], [6]. Thus, HomifyOne is introduced to fix these gaps.

1.2. Project Context

The UK new-build personalisation process may be unfamiliar to readers, so the following illustrative scenario establishes the domain. Mary and James are a young couple who have paid a deposit on their first home, a three-bedroom new-build house outside Leicester. While the property is being built, the property developer (also called the Site Manager) can allow them to personalise parts of the house before handover. These decisions may cover flooring, kitchen finishes, bathroom tiles, lighting, wardrobes, appliances and other interior products.

During the build, Mary and James receive an invitation to a Choices Appointment. This is the point at which buyers select products that will be installed before handover. In some developments, the purchase offer includes a personalisation budget or allowance. Instead of operating only as a reduction in the house price, this amount may be used towards personalising the home. In HomifyOne, choices or included choices are standard finishes already included in the home price, such as a standard kitchen colour, tile or carpet. Extras or paid upgrade products go beyond that range, for example a premium worktop, integrated appliances, additional sockets, wardrobes, upgraded flooring or premium lighting. The allowance can be applied to paid upgrades, and if the spending exceeds the allowance limit, the buyer would normally pay the difference. Payment collection is outside this project's scope.

Mary and James may have approximately 90 minutes to make decisions using catalogues and price lists, that will remain in their home for years. However, the process does not necessarily understand that they have a toddler and a dog. A pale carpet may look attractive in a catalogue while being futile to their household. Consequently, they may spend thousands of pounds to upgrades without personalised support or any form of guidance to decide them make a suitable purchase for their lifestyle within the allowance amount.

Their submission is only the beginning of the operational process. The property developer must review and approve the selections, identify the supplier for each product and generate purchase orders. Suppliers procure the approved items, provide delivery estimates and deliver them to the relevant plot. The developer and site team then coordinate installation before handing the completed home to the buyers.

This scenario shows why HomifyOne is not an ordinary e-commerce application. Every selection is attached to a reserved plot, house type, deadline and allowance, and approval must occur before supplier fulfilment. Industry evidence supports the need for clearer quality, communication and customer centred housebuilding processes [1], [3], while research shows that housing decisions are shaped by lifestyle, value and household needs [2], [4], [7]. In practice, catalogues, spreadsheets, email threads and disconnected supplier records can fragment and even disrupt the process at times. HomifyOne addresses this gap in an efficient manner.

1.3. Target users and their problems

HomifyOne caters to four different stakeholders facing unique and unaddressed problems.

- Firstly, the buyers, especially the first-time buyers may not have proper knowledge about choosing the right choices and paid extras for their new home. They have to select products from very large catalogues for which they do not get proper guidance. They may also not be aware of their budget or which products are suitable for their lifestyle. Since buying a home is a one-time permanent investment, buyers should get intelligent support based on their needs and preferences [4], [7].
- Secondly, the developers who often manage multiple buyers simultaneously also must handle many other chores which includes coordinating selection approvals, purchase orders, and supplier communication [8], [9]. However, in many cases, this work is split among email, spreadsheet and general project management tools which can make their work tedious and time-consuming.
- Thirdly, the suppliers receive unstructured order communication from the developer but have no dedicated portal for accepting orders, confirming their delivery timeline, and submitting invoices.
- Lastly, the administrator does not have a centralised governance layer to manage user accounts, maintain product data, and monitor platform activities in one place. Without such a system, it becomes difficult to control users, update product information, and oversee the smooth functioning of the platform.

1.4. Aim and Objectives

The main aim of this project is to design, build and evaluate HomifyOne as an efficient web application that replaces the current disjointed new-build personalisation and workflow process by offering a unified, intelligent digital experience for Buyers, Developers, Suppliers and Administrators.

To achieve this aim, the following objectives were set:

- To provide secure, role-based access for four distinct user types (Buyer, Developer, Supplier and Administrator).
- To help buyers select suitable personalisation products from the available standard choices and paid extras.
- To generate lifestyle-based recommendations for paid upgrades by capturing info from the buyer through a questionnaire and applying AI-driven semantic matching
- To give buyers a live view of their spending against their allocated extras allowance.
- To support buyer submission, developer approval or rejection with reasons, revision and top-up orders.
- To automatically generate purchase orders, grouped by supplier, upon developer approval.
- To enable suppliers to manage order fulfilment, delivery dates and invoices.
- To provide notification, calendar and communication facilities across all stakeholders.

1.5. Proposed Solution Overview

HomifyOne is a centralised, role-based web application that brings all four stakeholders into one platform. Rather than giving the buyers a catalogue to browse alone, it captures their personal preferences via a questionnaire and then uses AI-driven semantic matching to fetch the recommendations that fit to how the buyer intends to live in their home. A dynamic live pricing engine tracks how the buyer spends against their extras allowance in real-time. Besides that, HomifyOne manages the complete workflow process including developer review and approval, automated purchase order (PO) generation grouped by supplier, a dedicated portal for different suppliers, invoice management, and in-app notifications for all stakeholders at every workflow event. In addition to these, HomifyOne supports real-time collaboration among the users and also integrates various external APIs to provide the best experience for its users. It also includes AI capabilities to provide more personalised guidance and help buyers in making better decisions during the home personalisation process.

1.6. System Overview

1.6.1 Workflow Summary

The principal workflow begins with the plot creation and buyer allocation by the administrator. The developer sets a selection deadline, which serves as an implicit grant of access for the buyers. The buyer completes the questionnaire, reviews recommendations and standard choices, adds items to a basket and submits. The developer either rejects the submission with a reason or approves it. Approval generates supplier-grouped purchase orders. Each supplier acknowledges its purchase order, sets an Estimated Time of Arrival (ETA), marks it sent and fulfilled and uploads an invoice. Once supplier evidence is complete, the developer can notify the buyer that installation is starting.

The workflow diagram below (Figure 1) provides an overview of the application's overall flow.

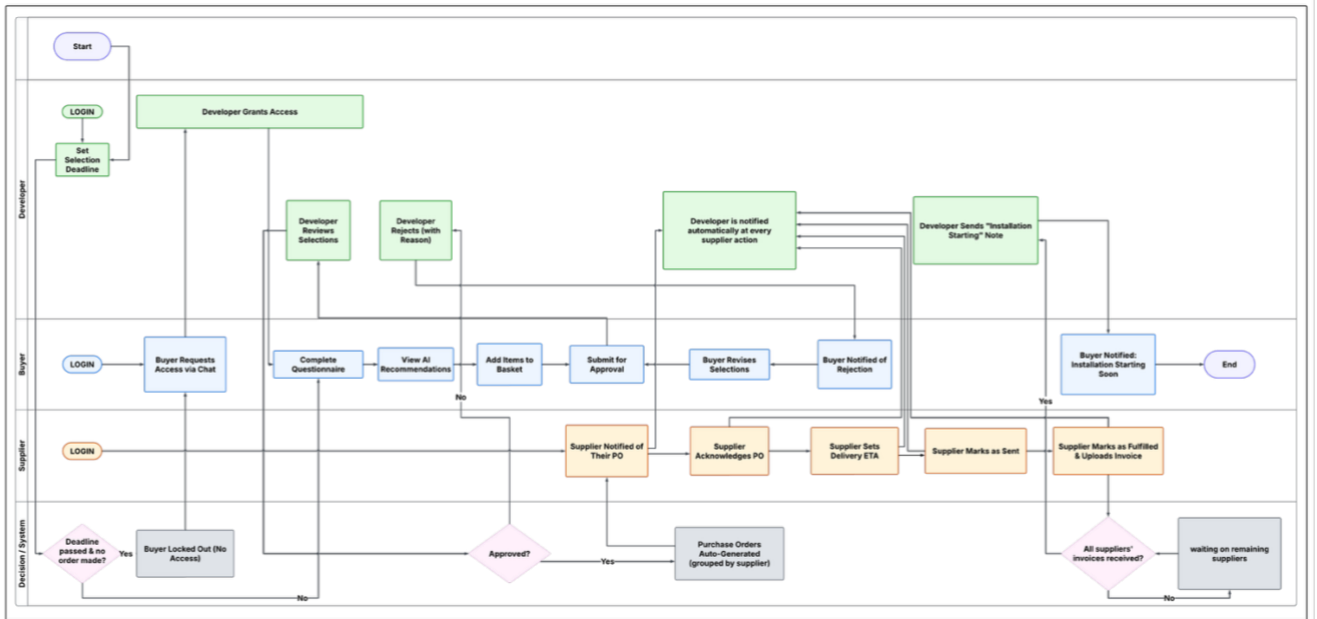


Figure 1. Workflow Diagram

Chapter 2. Background Research

2.1. New-build personalisation as a socio-technical workflow

Housing quality is not determined solely by structural compliance. Buyers experience quality through communication, expectation management, finishes, usability and the extent to which the completed property reflects promised personalisation [1]. The Competition and Markets Authority's housebuilding study similarly draws attention to consumer outcomes, information and the operation of the housebuilding market [3]. These concerns provide the broader context for HomifyOne i.e., a digital interface is valuable only if it improves the reliability of the underlying process.

The process has several socio-technical characteristics. First, it is multi-party. A buyer expresses preferences, a developer governs what is permitted for a plot, and one or more suppliers fulfil the resulting demand. Second, it is financially constrained. Some products are included in the agreed home price, whereas upgrades consume an allowance or create an additional expense. Third, it is stateful. A selection can be drafted, submitted, rejected, revised and approved before any supplier action occurs.

These characteristics mean that an ordinary catalogue model is not sufficient. A conventional shop optimises discovery, checkout and delivery to an individual customer. HomifyOne instead treats the plot as a shared operational anchor. The buyer may submit more than once, but only under controlled conditions. The developer's approval creates supplier-grouped purchase orders, and completion depends on multiple suppliers. This design turns a collection of user interfaces into a coordinated workflow.

2.2. Research Element 1: NLP-Based Semantic Recommendation Engine

HomifyOne's recommendation engine is the technically novel part of the system. This solves a problem that most property platforms haven't fixed. People naturally describe their needs in everyday words, like wanting to have a home for a young family or one that's easy to maintain. Yet most catalogues still rely on rigid categories and filters. The recommendation engine is designed to bridge that gap by interpreting buyer language and matching it to relevant catalogue items.

Instead of relying only on manual filtering or static categories, the system converts both buyer questionnaire answers and product details into semantic representations, then compares them by similarity. Sentence-BERT is well suited to this. Reimers and Gurevych showed that the model produces dense sentence embeddings that perform well on semantic similarity tasks, which makes it a good fit for matching what a buyer wants to what a product actually offers [10]. To search across these embeddings quickly as the catalogue grows, the system uses FAISS (Facebook AI Similarity Search), based on the method developed by Johnson, Douze and Jégou for fast similarity search over large embedding spaces [11].

These technologies also suit HomifyOne for a more specific reason. As a new system, HomifyOne does not yet have the volume of buyer interaction data that collaborative filtering normally requires to work well. A semantic, knowledge-based approach is more realistic at this early stage. This matches the wider literature on hybrid recommender systems, which shows that combining several strategies, including content-based matching, rules and similarity scoring, tends to work better than relying on just one [12].

The benefit for buyers is easy to see in practice. A buyer who describes their household as a busy family with young children and a dog could be guided towards durable, easy-clean, scratch-resistant finishes rather than decorative upgrades that may not suit daily use. In this way, the catalogue stops being a passive list and becomes a tool that actively supports the buyer's decision, based on how they actually live.

2.3. Research Element 2: Conversational AI Assistant

The second research element is a conversational AI assistant that is grounded in HomifyOne's catalogue. Its purpose is to let buyers ask natural language questions, for example “What flooring suits a high-traffic hallway under £800?” or “Which kitchen upgrades are useful for a family home?”, without depending entirely on developer brochures or manual sales advice for answers that could otherwise be generated automatically.

This is made possible through Retrieval-Augmented Generation (RAG), a technique introduced by Lewis et al. It works by combining a generative language model with retrieved external context, so that what the model produces stays grounded in real source material rather than being invented [13]. Guu et al. reach a similar conclusion with REALM, showing that retrieval-augmented language models are valuable for tasks that depend heavily on accurate, specific knowledge [14]. This supports the same grounding approach used here.

This grounding matters a great deal for HomifyOne. The assistant must not invent products, prices or specifications that are not in the live developer catalogue, because doing so could mislead a buyer making a significant financial decision. For this reason, the implementation compared two approaches, one a retrieval-only pipeline built with FAISS which simply returns ranked catalogue matches, and a full RAG pipeline which adds a Gemini-based conversational layer on top of retrieval.

Even with retrieval, hallucination can still occur if the generative model is not given proper instructions on how to use the retrieved content. Hence, Prompt engineering is required to control the assistant's behaviour, response and handle fallbacks. White et al. describe prompts as instructions that can enforce rules, automate processes and shape the quality of LLM output [19]. HomifyOne's Gemini prompt guides the assistant to stay within the retrieved catalogue content, mention product names and prices when available, and avoid answering questions about products not found in the catalogue. This reflects Zamfirescu-Pereira et al.'s finding that LLMs work better when prompts are designed systematically instead of being written as vague instructions [20]. Therefore, the assistant is designed as a catalogue-aware home personalisation advisor, helping to reduce hallucination risk and improve the reliability of its responses.

However, the final choice is based on four criteria:

1. Recommendation relevance
2. Response accuracy
3. Hallucination risk
4. Implementation feasibility

2.4. Theoretical Foundation for Lifestyle and Behavioural Preference Modelling

The lifestyle questionnaire in HomifyOne is based on established ideas from consumer behaviour and human-computer interaction. One important theory is lifestyle segmentation, mainly Plummer's Activities, Interests and Opinions model, which explains that consumer choices can be better understood by looking at how people live, what they value, and what practical needs influence their decisions [15]. This supports HomifyOne's use of buyer details such as household size, preferred interior style, room usage, storage needs, budget sensitivity, and lifestyle priorities.

This approach can also be related with Means-End Chain Theory which explains how consumers connect product attributes to practical benefits and personal values [16]. For example, durable flooring is not only a product choice, it can certainly represent easier maintenance, family practicality and long-term satisfaction. This supports HomifyOne's aim to recommend extras based on buyer preferences rather than simply displaying a static catalogue.

Furthermore, Choice Overload Theory justifies the need for AI-assisted filtering, as too many options can reduce decision confidence and satisfaction [17]. This is highly relevant to new-build personalisation, where buyers often face a large catalogue of upgrades within limited time and budget pressure. Finally, the Technology Acceptance Model also states that users are more likely to adopt a system when they feel it as useful and easy to use [18].

Therefore, HomifyOne's questionnaire, AI recommendation engine and simplified buyer journey are research-informed design decisions intended to improve confidence, usability and decision quality.

2.5. Existing Platform Gap Analysis

Only a few platforms currently provide digital support for new-build home personalisation in the UK. Housebuilder Pro offers a buyer-facing portal where developers upload product catalogues, and buyers browse choices and extras before submitting their selections [21]. Clifix provides tools for managing customer care and defects, along with limited communication features before completion [22]. ContactBuilder helps developers manage

buyer options through a CRM and extras portal, but it mainly supports selection rather than intelligent personalisation or procurement coordination [23]. The table below compares the publicly available features of selected platforms with the proposed scope of HomifyOne.

Features	Housebuilder Pro	Clixifix	ContactBuilder	HomifyOne
Buyer lifestyle preference capture	×	×	×	✓
AI-driven personalised product recommendations	×	×	×	✓
Conversational AI assistant	×	×	×	✓
Room-by-room selection workflow	✓	×	✓	✓
Real-time live budget tracker	×	×	×	✓
Product comparison tool	×	×	×	✓
Developer approval workflow	✓	×	✓	✓
Automated PO generation grouped by supplier	×	×	×	✓
Supplier fulfilment and invoice tracking	×	×	×	✓
Multi-role platform: Buyer, Developer, Supplier and Admin	×	×	×	✓
Real-time in-app messaging across stakeholders	×	✓	×	✓
PDF selection summary for buyers	×	×	×	✓

This comparison analysis confirms that no existing application combines AI-driven buyer preference intelligence, conversational guidance, procurement automation, and multi-stakeholder coordination in a single platform. Therefore, HomifyOne should be seen as a new type of tool, not just a better version of an existing one.

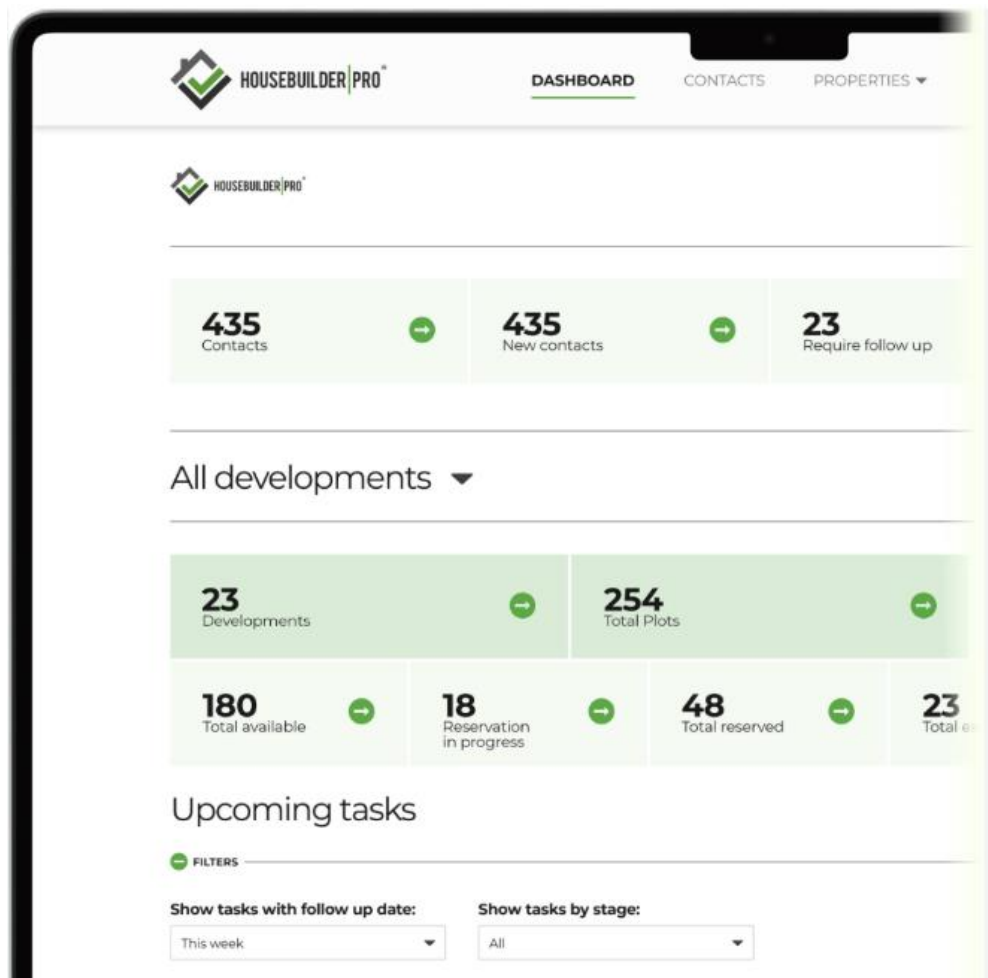


Figure 2. Developer Dashboard screenshot from Housbuilder Pro

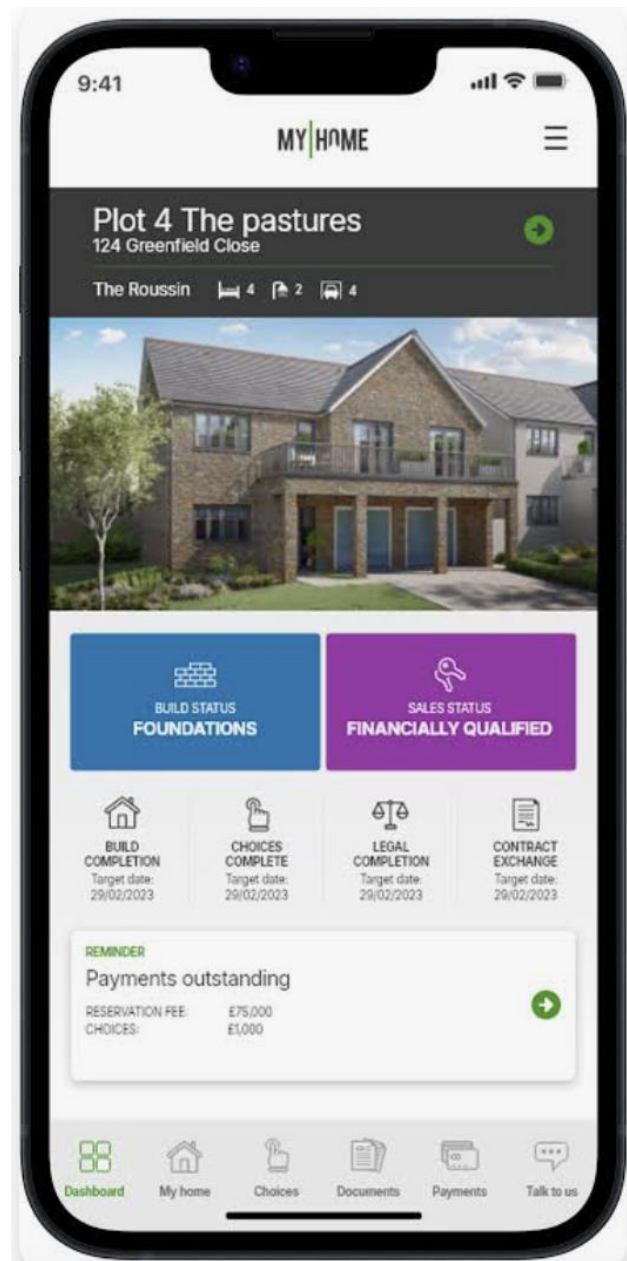


Figure 3. Buyer Dashboard screenshot from Housbuilder Pro

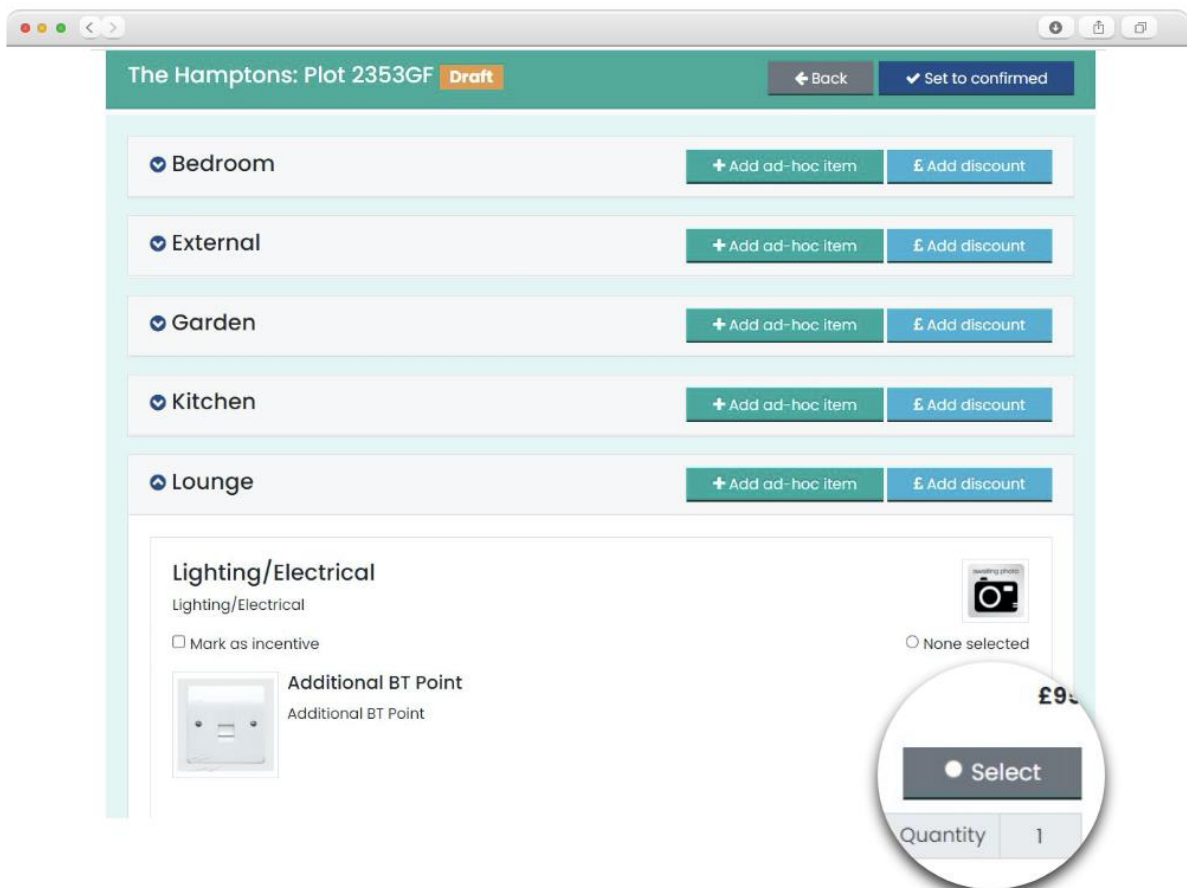


Figure 4. Buyer choices & extras selection portal from ContactBuilder application

Plots [+ New plot](#)

Plot number	Type	Style	Status	Price	Reservation	Active	
1	Detached	Bramhall	Available			✓	View
2	Detached	Carron	Available			✓	View
3	Detached	Brearley	Confirmed		Ariya Kouma	✓	View
4	Detached	Stanbury	Confirmed		Julianne Bailey	✓	View
5	Semi	Edenfield	Available			✓	View
6	Semi	Edenfield	Completed		Felicity Stowe	✓	View
7	Detached	Stanbury	Confirmed		John Hawke	✓	View
8	Detached	Stanbury	Available			✓	View
21	Semi	Edenfield	Available			✓	View

Figure 5. Developer Dashboard screenshot from ContactBuilder application

Chapter 3. Requirements

3.1. Essential Requirements

- Multi-role secure authentication for Buyer, Developer, Supplier, and Admin using JWT and bcrypt password hashing - Users log in through a single interface. Passwords are hashed with bcrypt before storage and a signed JWT is issued on login, carrying the user's role and ID to authenticate every subsequent API request.
- Role based access control and dashboards - Each role has a dedicated dashboard showing only the data and actions relevant to them. Access restrictions are enforced at the API level.
- Admin management of user accounts and product catalogue - Admins can create, edit and delete user accounts across all roles. They also manage the master product catalogue, including adding and updating products.
- Buyer personalised plot dashboard showing property details, selection status and deadline countdown - Each buyer sees their plot details, property address, and an overview of their selection progress across all rooms. A countdown timer displays the number of days remaining before their selection deadline.
- Room-by-room choices portal to browse and select standard choices for Buyers - Buyers navigate through their property room by room, browsing and selecting standard choices available for their specific plot type. Selections can be added, swapped, or removed at any point before submission.
- Buyer lifestyle questionnaire and AI recommendation engine for extras selection - Buyers complete a short questionnaire covering their lifestyle preferences. Responses are processed through sentence-transformer embeddings and FAISS semantic search pipeline to return a ranked list of personalised extras recommendations.
- Dynamic live pricing engine that recalculates the total extras cost as selections are added, removed or changed - As buyers add, remove, or change extras selections, the total cost recalculates instantly. The running total is displayed persistently on the buyer dashboard and alerts the buyer if their selections exceed their allocated budget.
- Developer sets deadline for Buyer selection, reviews, approves/rejects Buyer's submission - Developers assign a selection deadline to each buyer. Once a buyer submits, the developer reviews and either approves it which in turn trigger purchase order generation or rejects it with a written reason for revision.
- Automated purchase order generation and grouping by supplier - On approval, the system automatically generates purchase orders, consolidating all selected items from the same supplier into a single PO.

- Supplier restricted PO portal where they can view, acknowledge POs, update status and set ETA - Suppliers see only the purchase orders assigned to them. They can acknowledge receipt, update fulfilment status, and set an estimated delivery date.
- Supplier invoice upload against assigned purchase orders - Suppliers upload invoices directly against a specific PO within the platform, which keeps each invoice linked to its corresponding order for direct developer review.
- Developer PO and invoice management dashboard - Developers have a consolidated view of all purchase orders and supplier invoices. They can monitor fulfilment status, review uploaded invoices, and manage them as required.
- In-app notifications for key workflow events - Real-time notifications are delivered when important actions occur i.e., when a buyer submits selections, a developer approves or rejects, or when a supplier acknowledges a PO.

3.2. Desirable Requirements

- Real-time in-app messaging for relevant user communication using WebSocket connections via Socket.io - Buyers and developers can message each other directly within the platform. Conversations are persisted in the database and use the existing Socket.io infrastructure with dedicated chat rooms per Buyer-Developer pair.
- Calendar synchronisation for scheduled Buyer-Developer meetings and Buyer selection deadlines - When a developer sets a selection deadline or schedules a meeting, the platform creates corresponding events in both parties' Calendars via the internal Calendar feature.
- Supplier ETA synchronisation to Developer's Calendar - When a supplier sets or updates a delivery date on a PO, the platform automatically syncs the date as a calendar event in the developer's Google Calendar.
- Developer interactive map displaying all assigned development location pins with a drill-down panel listing plots, buyers, and their selection status - Developers managing multiple sites can view all developments as location pins on an interactive map. Clicking a pin opens a side panel listing all plots, assigned buyers, and their current selection status.
- Standard choices comparator for Buyers - Buyers can select two or more standard choices within the same category, view them side by side and comparing their attributes to support more informed decision
- AI Buyer assistant for product selection and conversational guidance - An embedded conversational assistant lets buyers ask natural language questions about catalogue products. This will include a set of suggested questions that buyers can select to quickly ask common queries

- PDF selection summary generated on approval, downloadable by Buyer and Developer - When a submission is approved, the platform generates a PDF of the buyer's complete selections. This is downloadable by both buyer and developer.

3.3. Optional Requirements

- AI invoice summariser to extract key information from uploaded supplier invoices - When a supplier uploads an invoice PDF, the platform extracts key fields using an AI pipeline which in further reduces manual data entry for developers.
- Supplier invoice history and payment status dashboard showing all submitted invoices with paid/pending status updated by the Developer - Suppliers can view all invoices they have submitted along with the payment status updated by the developers
- Admin analytics view - This feature provides platform-wide activities that are worth capturing in the form of charts and plots, which further act as critical KPIs for the application to function seamlessly.
- Email notifications for all key workflow events as a fallback to in-app notifications for all user roles - Along with in-app notifications, the platform sends email notifications for all key workflow events, thus making sure that users are informed even when they are not actively logged into the platform.

3.4. New Features Added

- Top-up ordering model - This lets buyers submit additional selections after an existing order is approved or rejected, rather than being blocked entirely
- Promo code and personalised credit system - Upon completing the lifestyle questionnaire, buyers receive an automatic cash-off credit (£50-£250, calculated as 3% of their extras budget). The credit is deducted first, and a 10% promo discount is then applied to the remaining balance. To prevent fraud, server-side history verifies that first-order-only rewards are applied strictly once. This gamification strategy successfully incentivizes questionnaire completion.
- Buyer portal enhancements - Included a Selection summary page that lists all past orders with downloadable selection summary PDFs. On the buyer dashboard, added an order progress tracking bar.
- Delivery status notification workflow - Once every supplier invoice for a plot has been submitted, the developer can send a text progress update directly to the buyer via in-app notification. This feature will be enabled only when all the suppliers deliver the order items to the plot site and submit their invoices in their respective portals.

3.5. Non-functional requirements

- Security requirements included password hashing, token verification, server-side role checks, supplier data isolation, safe file access and validation of user input.
- Usability requirements included role-specific navigation, visible deadline and status feedback, readable price breakdowns, responsive pages and explanations for recommendations.
- Maintainability requirements included separation of client, server and AI service, feature-oriented routes and controllers, reusable contexts, and schema-based persistence.
- Reliability requirements included fallback email, stored notifications where real-time delivery is unavailable, and durable document storage.
- Performance requirements included paginated catalogue presentation, precomputed product embeddings and bounded FAISS retrieval.

3.6. Data and privacy requirements

- The application processes names, contact details, plot assignments, questionnaire preferences, messages, orders and invoices. To ensure data minimization, each role receives only the information required to perform its responsibilities. These restrictions were enforced through server-side authentication, role-based authorisation and ownership checks rather than relying only on information being hidden in the React interface. Buyer-specific browser data, including cached baskets and recommendations, was cleared when a different buyer logged in on the same device to prevent information from being carried between accounts.
- Access to invoices and message attachments was controlled through authenticated endpoints and time-limited URLs instead of publicly exposing object-storage paths. Credentials and third-party service keys were stored in environment configuration rather than application source code. These measures reduced unauthorised access and accidental exposure of sensitive information.

3.7. Requirement changes

- Several changes emerged during implementation. The original one-order assumption was replaced by a top-up ordering model, allowing buyers to make additional selections after an approved or rejected order while preventing a second submission during pending review. A personalised credit and promo-code mechanism was added to encourage questionnaire completion. PDF generation changed from Puppeteer to PDFKit, Redux Toolkit was replaced by React Context, Cloudinary was limited to product images while AWS S3 stored documents, and Google Calendar OAuth was replaced by an internal calendar.
- These changes preserved the project aim. They reduced unnecessary infrastructure while increasing domain fidelity. Particularly, the top-up model corrected an unrealistic

workflow assumption that the real buyers may make additional choices after an earlier decision. The cost was additional complexity in cumulative allowance and per-order pricing, which had to be calculated separately to avoid double counting.

Chapter 4. Technical Specification

4.1. Tech Stack Overview

Category	Technology	Purpose
Frontend	React	JavaScript library for building client-side user interface
	HTML 5	Structure the content and layout of web page
	CSS3	Style and format web pages
	Tailwind CSS	Utility-first style framework for responsive UI development
	Context API	Manage application state across components
	Axios	Handle HTTP requests from the frontend to backend
Backend	Node.js	Run JavaScript code on the server side
	Express.js	Create RESTful APIs and handle server-side routing
AI Microservice	Python	Programming language for AI service logic
	FastAPI	Build fast and lightweight API services in python
Database & ODM	MongoDB Atlas	Store all application data in a flexible, cloud-hosted document database
	Mongoose	Define schemas and interact with MongoDB from the Node.js backend
Authentication	JWT (JSON Web Tokens) & bcrypt	Manage secure authentication and password protection
Real-time features	Socket.io	Real-time communication between users
Media File Storage	Cloudinary & AWS S3	Store, manage and retrieve media files in the cloud
Email	Nodemailer & SendGrid	Send OTP password reset emails and event notification emails
PDF Generation	Puppeteer (Node.js)	Generate server-side PDF selection summaries upon developer approval
Third-party integration	Leaflet.js	Display interactive maps in the web application
AI-Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Convert text into semantic vector representations for similarity matching

Vector Search	FAISS	Perform fast cosine similarity search over product embeddings
LLM (optional)	Google Gemini API	For generative AI or conversational AI capabilities
API Validation	Postman	Manually test and validate all API endpoints
Task Scheduling	nodecron	Schedule automated background tasks
Testing Frameworks	Vitest, Jest, Supertest, Playwright	Frameworks to test JavaScript code and backend API endpoints
Version Control	GitLab	Manage version control
IDE	Visual Studio Code	Code editor

Chapter 5. Development Methodology and Project Management

5.1. Overview of Development Methodology

Homifyone followed an adapted waterfall structure with iterative implementation. Requirements, background research and a technical specification were established early and then the major milestones followed the business sequence: authentication and data models, buyer choices and recommendation, developer review, procurement, collaboration and governance, testing and reporting. Within each milestone, implementation was iterative. Test findings and workflow demonstrations caused local changes without discarding the overall plan.

Given the university's assessment schedule, this approach was suitable for an individual academic project. Moreover, an agile model would not have been helpful since the project is not operating with any external team or client. A rigid waterfall process would have made it difficult to correct the one-order assumption or substitute the calendar design. This proves that the adapted model provided a stable scope while allowing me to revise as and when the project needed.

5.2. Prioritisation and milestones

Work was prioritised according to dependency and risk. Requirements were categorized as essential, desirable and optional based on this. The primary focus was to complete the essential requirements before moving forward with other features. Meanwhile, the features related to the administrator role was deliberately pushed to the later phase of implementation as it directly does not have any impact on the other roles' tasks. The procurement pipeline was completed earlier than first planned, and this in turn provided some time to include top-up ordering, promo credit, delivery status notification workflow features to be included in the implementation. This way of prioritising work enabled me to do multiple rounds of testing manually and otherwise as planned.

5.3. Tools and Configuration Management

Git was used throughout the project for version control, which allowed implementation changes and development progress to be tracked systematically. The application was organised into separate React client, Express server, FastAPI AI service and Playwright end-to-end testing components. Each component included its own package manifest or requirements file to define and manage its dependencies.

Environment variables were used to configure the MongoDB database connection, JWT secrets, storage services, email functionality and Gemini API access without hard-coding sensitive values into the application source code. This separation made the individual components easier to develop, maintain and test independently. However, it required the three runtime services and their external integrations to be configured consistently when running the complete application.

Chapter 6. System Design

6.1. System Design Architecture

The following diagram represents a high-level system design architecture of HomifyOne,

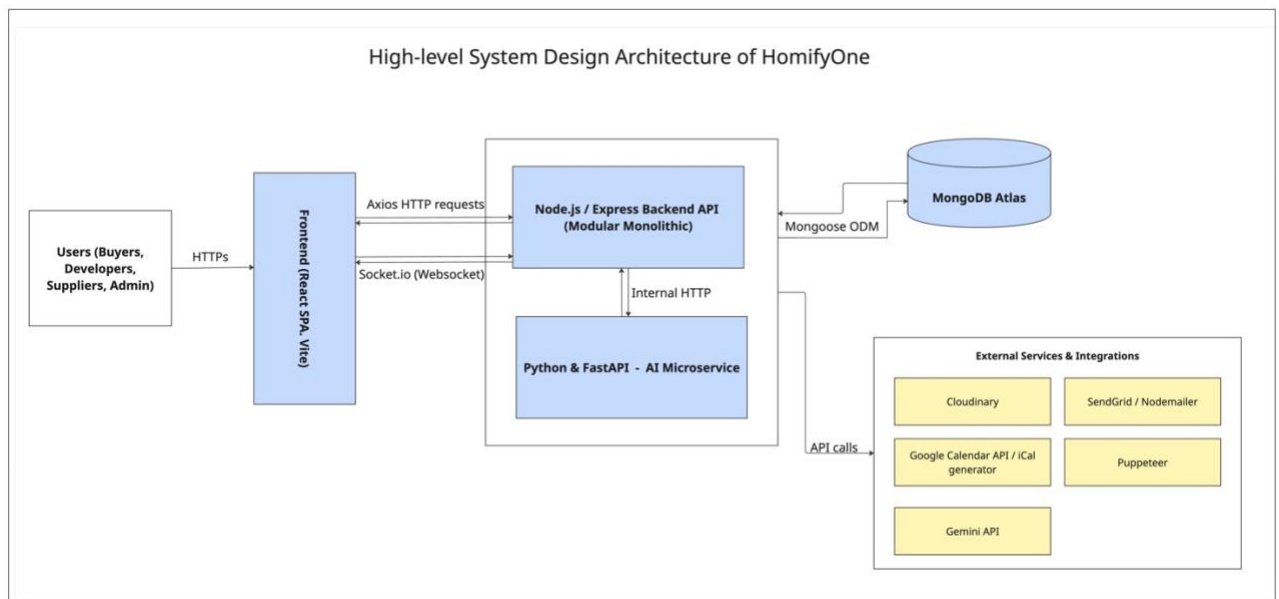


Figure 6: High-level System Design Architecture

6.2. Backend architecture

The backend is built as a modular monolithic application using Node.js and Express.js, internally structured using a layered MVC-style approach. Every incoming request is processed through shared middleware before reaching its route handler. For AI operations, the Node.js backend delegates to a separate Python microservice built with FastAPI, via internal HTTP. This separation keeps the AI logic and its dependencies isolated from the main application server. This way, the Node.js layer remains lightweight and the two services can be scaled or updated independently.

6.2.1. Middleware Stack

Middleware	Purpose
helmet	Sets secure HTTP response headers
cors	Restricts cross-origin requests to the frontend origin

express.json()	Parses incoming JSON request bodies
morgan	HTTP request logging in development
verifyToken	Validates the JWT from the Authorization header on every protected route
express-validator	Validates and sanitises request body fields per route
errorHandler	Centralised error response formatter

6.2.2. Backend Endpoints

Method	Endpoint	Purpose
/api/auth		
POST	/login	Authenticate and return signed JWT
POST	/logout	Clear token from cookies
POST	/forgot-password	Send password reset OTP via email
POST	/reset-password	Validate OTP and update password
/api/buyer		
GET	/dashboard	Fetch plot details, selection status, deadline
POST	/questionnaire	Submit lifestyle questionnaire responses
GET	/recommendations	Retrieve AI-ranked extras recommendations
GET	/pricing	Get live recalculated extras total
POST	/submit	Submit full selection for developer review
GET	/summary	Retrieve approved selection summary & PDF link
/api/developer		
GET	/dashboard	Fetch all plots, buyers, and their statuses
GET	/buyers	List all buyers assigned to the developer
GET	/buyers/:id/submission	View a specific buyer's submitted selections
POST	/buyers/:id/approve	Approve buyer submission and trigger PO generation
POST	/buyers/:id/reject	Reject submission with written reason
PUT	/buyers/:id/deadline	Set or update buyer selection deadline
GET	/purchase-orders	List all generated purchase orders
GET	/purchase-orders/:id	View a specific PO in detail
GET	/invoices	List all supplier invoices
PUT	/invoices/:id/status	Mark invoice as paid or flagged
GET	/map	Fetch all development locations with plot and buyer data
/api/supplier		
GET	/purchase-orders	List all POs assigned to this supplier
GET	/purchase-orders/:id	View a specific PO in detail

PUT	/purchase-orders/:id/acknowledge	Acknowledge receipt of a PO
PUT	/purchase-orders/:id/status	Update PO fulfilment status
PUT	/purchase-orders/:id/eta	Set or update estimated delivery date
POST	/purchase-orders/:id/invoice	Upload an invoice against a PO
GET	/invoices	View all submitted invoices with payment status
/api/admin		
GET	/users	List all users across all roles
PUT	/users/:id	Update user details or role
GET	/catalogue	View full product catalogue
POST	/catalogue	Add a new product
GET	/analytics	Fetch platform-wide analytics data
Python Microservice layer - /api/ai		
POST	/recommend	Accept questionnaire responses and return ranked product recommendations
POST	/chat	Accept buyer message and return response
POST	/summarise-invoice	Accept invoice PDF text and return extracted key fields

6.3. Frontend architecture

The frontend will be developed as a single-page-application using React, with Vite used as the build tool. After a user log in, the system will read their role from the JWT and redirect them to the correct dashboard using React Router DOM. Axios will be used to handle backend communication. A shared Axios instance will also be used such that the JWT Authorization header is included in every API request.

For real-time features, the frontend will use a Socket.io client connection. This connection will be created when the user logs in and closed when the user logs out. Redux Toolkit or Context API will be used to manage data and states while local component states will handle smaller UI related data. Tailwind CSS will be used across the frontend to create a responsive layout and utility-based styling.

6.4. Database Design

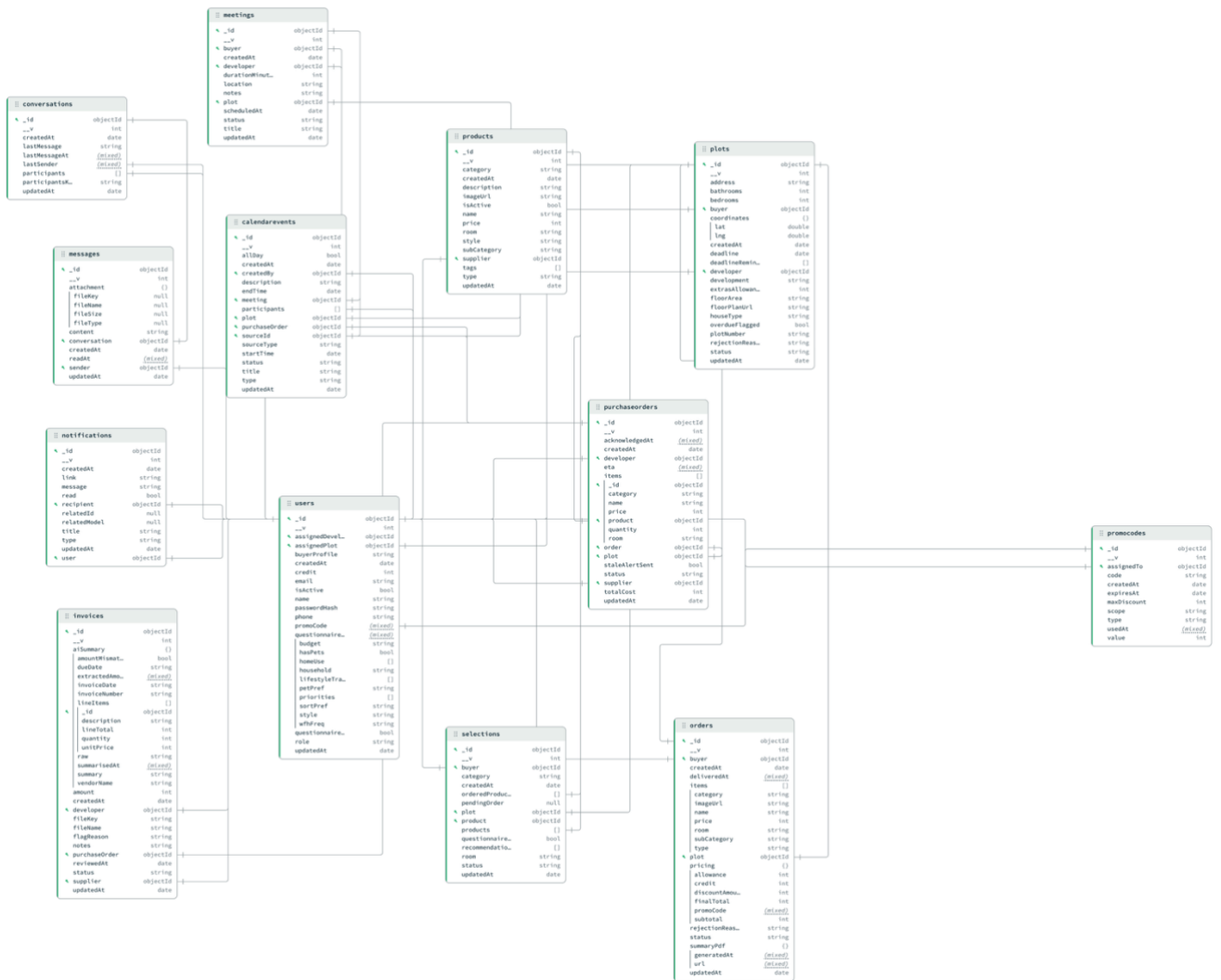


Figure 7: Database Design of Homifyone

6.5. Use case Diagram

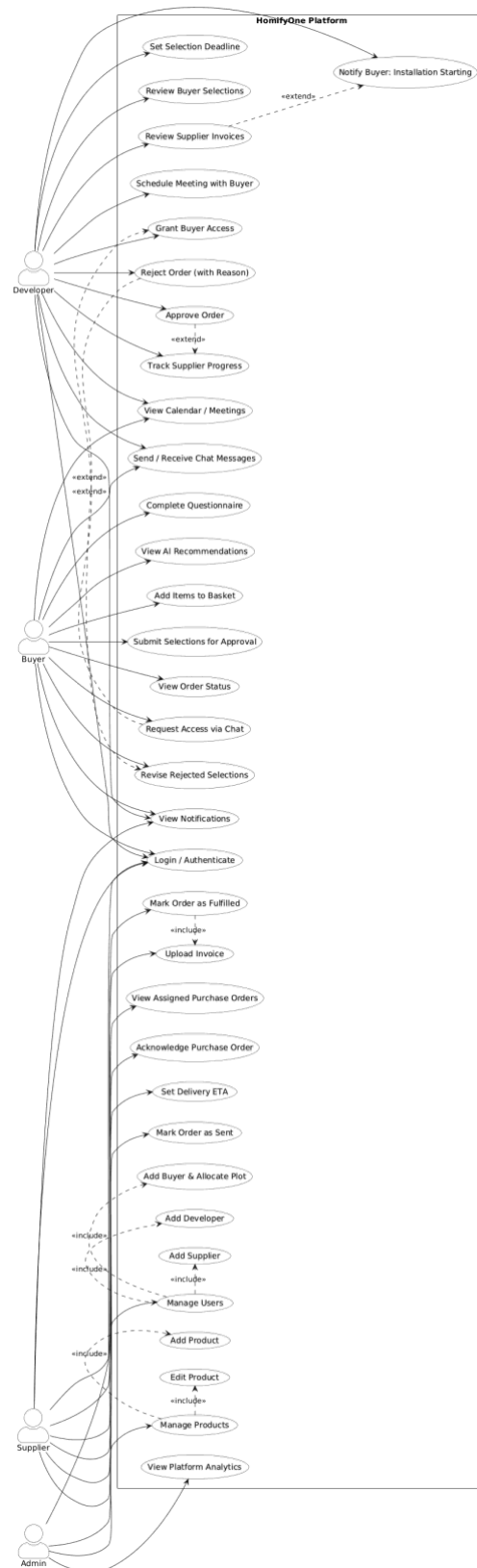


Figure 8: Use Case Diagram of Homifyone Platform

6.5.1. Specific Use Case - Product Ordering and Delivery to Site

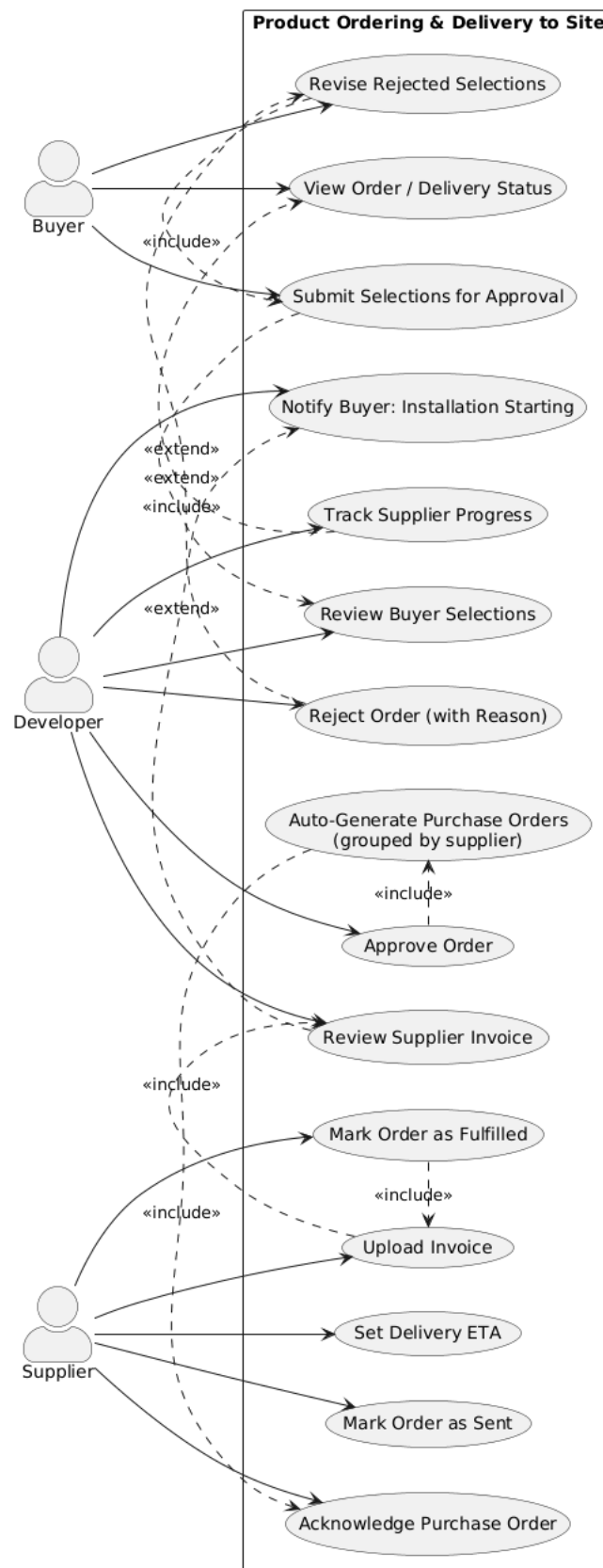


Figure 9: Use Case Diagram of product ordering and delivery to site

6.6. Event Lifecycles

6.6.1. Plot state transition from creation to completion

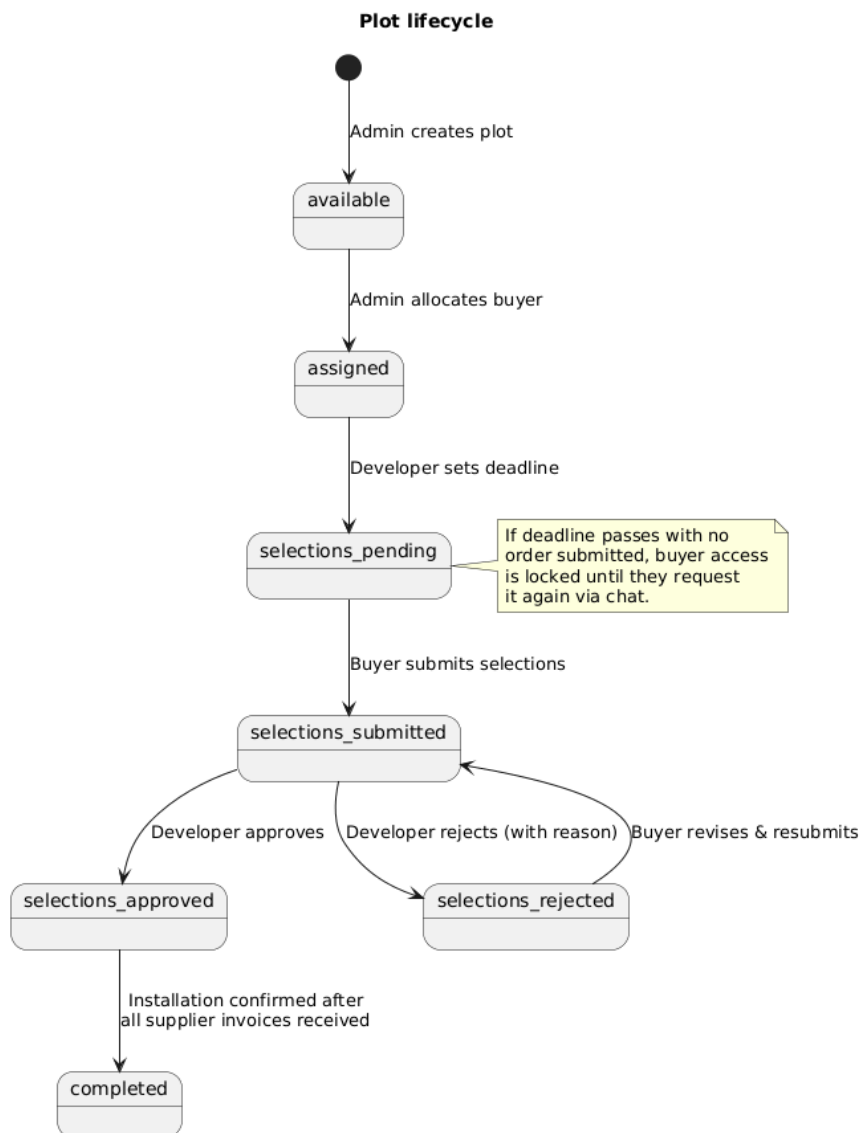


Figure 10: State Diagram of Plot lifecycle

6.6.2. Order and purchase-order lifecycles

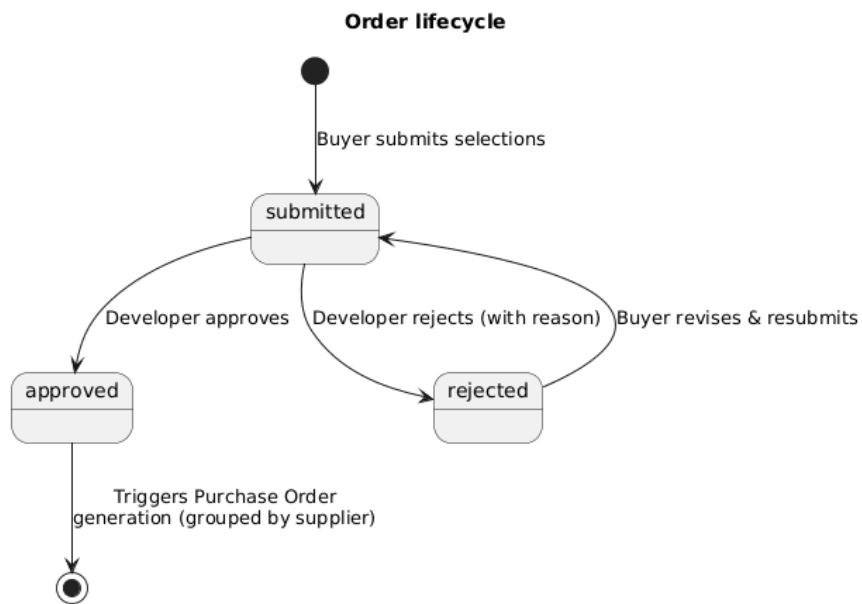


Figure 11: State Diagram of Order lifecycle

PurchaseOrder lifecycle

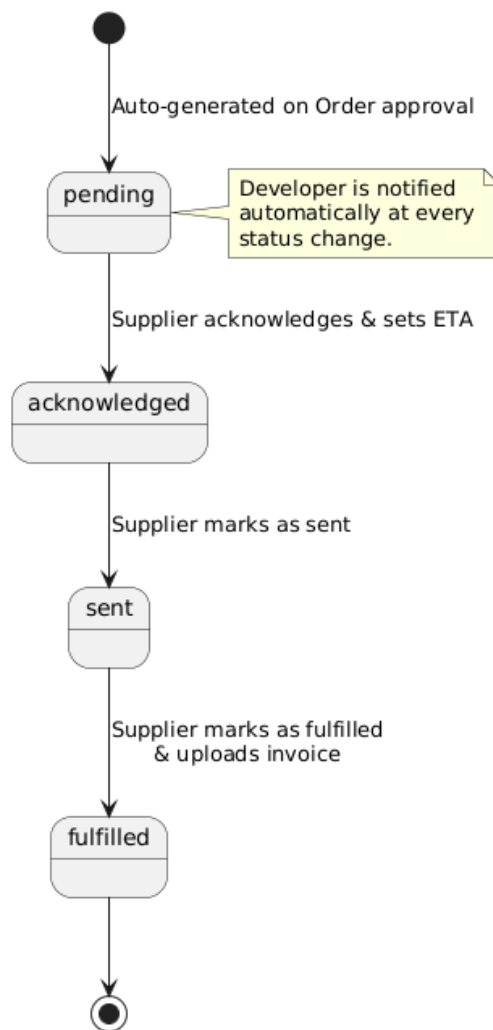


Figure 12: State Diagram of Purchase Order lifecycle

6.6.3. Recommendation Flow

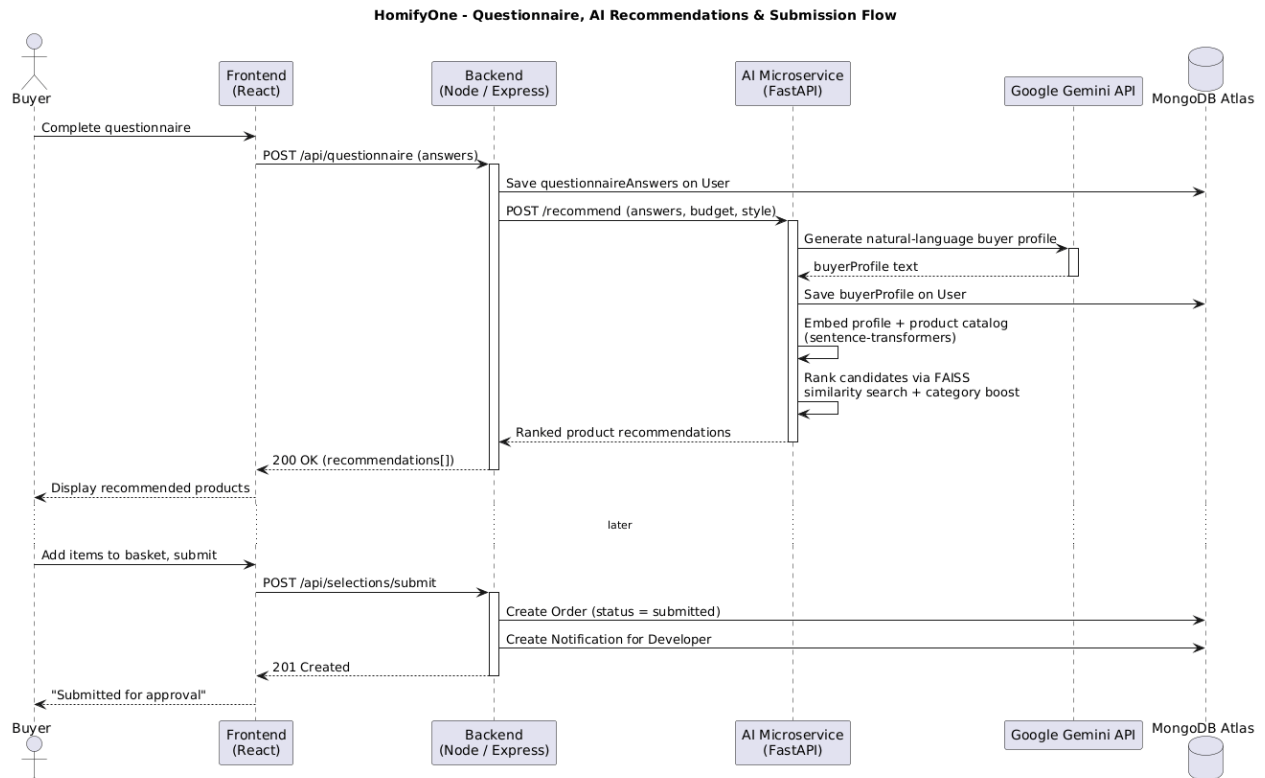


Figure 13: Sequence Diagram of Recommendation Flow

6.7. User interface and navigation design

6.8. Pagination and filtering

6.9. Calendar, messaging and notification design

6.10. Design of Auto-generation of Purchase Orders

6.11. Creation of Embeddings and storage for Recommendation Engine

6.12. Design of AI Assistant - Ask Homifyone

Chapter 7. Implementation

7.1. Authentication and Authorisation

7.2. Standard choices and paid upgrades

7.3. Questionnaire and profile building

7.4. Embedding preparation and FAISS Index

7.5. Hybrid scoring and explanations

7.6. Basket, credit and promotional pricing

7.7. Submission, approval and top-up orders

7.8. Purchase orders and supplier portal

7.9. Notifications, chat and scheduled work

7.10. AI assistant and summarisation of orders

7.11. Uploading invoice and summarisation

7.12. Administrator features and analytics

Chapter 8. Testing

8.1. Evaluation strategy

8.2. Unit Testing

8.3. Integration Testing

8.4. End To End Testing

8.5. Functional and Workflow Testing

8.6. API Testing

8.7. Evaluation of Recommendation output

Chapter 9. Critical Reflection

9.1. Key Achievements

9.2. Modified objectives and scope control

9.3. Technical challenges

Delivering HomifyOne presents the following key technical challenges:

1. **NLP-driven recommendation pipeline:** I have not previously worked with NLP or semantic similarity systems. Implementing the recommendation engine requires me to research how sentence-transformer models such as all-MiniLM-L6-v2 convert text into vector representations, how FAISS performs cosine similarity search over those vectors, and how buyer's responses can be mapped to ranked extras recommendations per room. The challenge is not just associated with the technical integration but to understand how to create product and query embeddings to generate recommendations based on buyer's lifestyle preferences.
2. **Designing a Multi-Role Event-Driven Workflow:** Designing a system where one user action triggers a series of state transitions, automated document generation, and notifications across different roles is challenging. To address this, I'll analyze the use of event-driven architectural patterns, design a state machine diagram, and implement structured error handling for the smooth functioning of the system.
3. **No Prior Experience with Conversational AI:** Building an AI assistant is a new area for me, and I need to research to select the right approach. I will compare a lightweight embedding-based method using Sentence-Transformers and FAISS with a full RAG pipeline using the Gemini API, and choose the option that is best and feasible.
4. **Security and Role-Based Access Control:** Enforcing strict data boundaries across four user roles while maintaining a smooth user experience is a security challenge. I will implement RBAC with JWT authentication and bcrypt password hashing, apply middleware-level role validation on every API endpoint, and conduct access control testing to verify that users cannot access another role's data.

9.4. Risk Control

- **AI/NLP unfamiliarity:** One of the primary risks was the need to work with technologies and concepts I had limited experience with, mainly the AI and NLP libraries, sentence-transformer embeddings, and FAISS for similarity search. To manage this, dedicated research time was planned in the earlier weeks of the schedule. This preparation proved sufficient, so no fallback was needed, and the

recommendation pipeline was completed as planned..

- **Fallback for Google Calendar API:** Integrating the Google Calendar API for Buyer-Developer meetings and Supplier Estimated Time of Arrival (ETA) synchronisation could be time-consuming due to the initially planned OAuth 2.0 configuration. Hence, an internal calendar was developed across the user portals.
- **Fallback for real-time notifications:** The system will use both real-time in-app notifications through Socket.io and email notifications through Nodemailer as a fallback. This ensures that important workflow events are communicated to the right stakeholder even if one notification method fails.
- **Ambitious Scope and Feature Creep:** The project scope grew beyond the original plan as development progressed, with four new features added to cover practical gaps that became clear once buyer, developer and supplier workflows were built out. To keep the project on track, this was managed through proper reprioritisation. I pulled the Milestone 5 forward while pushing the admin functionality back as part of reprioritisation.
- **Database Schema Design and Data Integrity:** Given the number of interlinked collections across buyer, developer, supplier and admin roles, schema design and backup practices were treated as an ongoing safeguard throughout development rather than a one-off task completed at the start.
- **Client-Side State Management:** A cross-buyer data leakage issue which surfaced during testing highlighted a broader pattern in how client-side state was being managed, rather than fixing it as a single bug.
- **Testing Limitations:** Automated end-to-end (E2E) testing was deliberately scoped to five critical user journeys instead of having a full coverage. E2E tests are the slowest and most expensive to run since each one uses a real browser, server and database, so a standard practice is to keep E2E test suites small and reserve them for important paths and make sure that edge cases and business rules are covered by unit and integration tests.

9.5. Architectural decision reflection

9.6. Evaluation reflection

9.7. Project management reflection

9.8. Professional and personal learning

Chapter 10. Conclusion

Chapter 11. Recommendation for Future Work

Chapter 12. References

- [1] The Housing Forum, *Building Homes Better: The Quality Challenge*. London, U.K.: The Housing Forum, Jun. 2023. [Online]. Available: <https://housingforum.org.uk/reports/key-publications/building-homes-better-the-quality-challenge>. [Accessed: Jun. 17, 2026].
- [2] Commission for Architecture and the Built Environment, *What Home Buyers Want: Attitudes and Decision Making Among Consumers*. London, U.K.: CABE, 2005. [Online]. Available: https://www.mae.co.uk/assets/pdfs/151123_Mae_Architects_What_Home_Buyers_Want_Ho_using_Architecture.pdf. [Accessed: Jun. 17, 2026].
- [3] Competition and Markets Authority, *Housebuilding Market Study: Final Report*. London, U.K.: CMA, Feb. 26, 2024. [Online]. Available: https://assets.publishing.service.gov.uk/media/65d8baed6efa83001ddcc5cd/Housebuilding_market_study_final_report.pdf. [Accessed: Jun. 17, 2026].
- [4] M. Schoenwitz, J. Gosling, M. Naim, and A. Potter, “How to build what buyers want: Unveiling customer preferences for prefabricated homes,” in *Proc. 29th Annu. ARCOM Conf.*, Reading, U.K., Sep. 2013, pp. 435–444.
- [5] EyeSite View, “How housebuilders can digitise the home buying journey,” *EyeSite View Blog*, Sep. 2025. [Online]. Available: <https://eyesiteview.co.uk/how-house-builders-can-digitise-the-home-buying-journey>. [Accessed: Jun. 17, 2026].
- [6] The Builders Daily, “2025: A turning point for homebuilding tech integration,” *The Builders Daily*, Feb. 2025. [Online]. Available: <https://www.thebuildersdaily.com/2025-a-turning-point-for-homebuilding-tech-integration>. [Accessed: Jun. 17, 2026].
- [7] M. A. Islam *et al.*, “Understanding high-involvement product purchase through an innovative machine learning approach: A case of housing type choice,” *Journal of Consumer Behaviour*, vol. 21, no. 6, pp. 1–18, 2022, doi: 10.1002/cb.2055.
- [8] X. Itämäki, “Streamlining the purchase order invoice process,” M.S. thesis, Tampere Univ. Applied Sciences, Tampere, Finland, 2025. [Online]. Available: <https://www.theseus.fi/handle/10024/885104>. [Accessed: Jun. 17, 2026].
- [9] ProcurePro, “The top 12 problems facing procurement in the construction sector,” *CPO Strategy*, Mar. 2025. [Online]. Available: <https://cpostrategy.media/blog/2025/03/04/the-top-12-problems-facing-procurement-in-the-construction-sector>. [Accessed: Jun. 17, 2026].
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3980–3990, doi: 10.18653/v1/D19-1410.
- [11] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021, doi: 10.1109/TBDATA.2019.2921572.

- [12] R. Burke, “Hybrid recommender systems: Survey and experiments,” *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002, doi: 10.1023/A:1021240730564.
- [13] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>. [Accessed: Jun. 19, 2026].
- [14] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proc. Int. Conf. Machine Learning*, 2020, pp. 3929–3938. [Online]. Available: <https://proceedings.mlr.press/v119/guu20a.html>. [Accessed: Jun. 19, 2026].
- [15] J. T. Plummer, “The concept and application of life style segmentation,” *Journal of Marketing*, vol. 38, no. 1, pp. 33–37, 1974, doi: 10.1177/002224297403800106.
- [16] J. Gutman, “A means-end chain model based on consumer categorization processes,” *Journal of Marketing*, vol. 46, no. 2, pp. 60–72, 1982, doi: 10.2307/3203341.
- [17] S. S. Iyengar and M. R. Lepper, “When choice is demotivating: Can one desire too much of a good thing?” *Journal of Personality and Social Psychology*, vol. 79, no. 6, pp. 995–1006, 2000, doi: 10.1037/0022-3514.79.6.995.
- [18] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989, doi: 10.2307/249008.
- [19] J. White et al., “A prompt pattern catalog to enhance prompt engineering with ChatGPT,” arXiv preprint arXiv:2302.11382, 2023. [Online]. Available: <https://arxiv.org/abs/2302.11382>. [Accessed: Jun. 19, 2026].
- [20] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, “Why Johnny can’t prompt: How non-AI experts try and fail to design LLM prompts,” in *Proc. CHI Conf. Human Factors in Computing Systems*, 2023, doi: 10.1145/3544548.3581388.
- [21] Housebuilder Pro, “Buyer choices & extras: Upgrade management,” Housebuilder Pro. [Online]. Available: <https://housebuilderpro.co.uk/buyer-choices-and-extras/>. [Accessed: Jun. 20, 2026].
- [22] Clixifix, “Housebuilders,” Clixifix. [Online]. Available: <https://www.clixifix.com/our-solutions/housebuilders/>. [Accessed: Jun. 20, 2026].
- [23] ContactBuilder, “Housebuilder CRM software,” ContactBuilder. [Online]. Available: <https://www.contact-builder.co.uk/>. [Accessed: Jun. 20, 2026].

Chapter 13. Appendices